

视频识别及物体/人脸识别

代码实现需求文档

基于数字图像与视频处理原理的工程实现

参考教材：《数字图像与视频处理》 第4-6、9-11章

技术栈：Kotlin + OpenCV 4.x + JNI + Android MediaCodec

项目：OpencvProcessing (Android 图像处理实验平台)

日期：2026年9月

第一章 文档概述

1.1 编写目的

本文档旨在为"视频识别及物体/人脸识别"功能模块的代码实现提供详尽的需求规范与架构设计指引。文档基于《数字图像与视频处理》教材第4章（图像分割）、第5章（压缩编码原理-运动估计）、第6章（压缩编码标准-H.264/HEVC运动补偿）、第9章（质量评价-运动信息提取）、第10章（基于内容的图像和视频检索）和第11章（图像识别-SVM/CNN）的理论知识，结合项目中已有的imagerecognition、imageCVdeal等模块代码基础，定义新模块的实现原理、代码架构、接口规范和测试方案。

1.2 项目背景

OpencvProcessing项目是一个围绕《数字图像与视频处理》教材实现的Android图像处理实验平台，由6个Gradle模块组成：app（UI路由层）、imageCVdeal（OpenCV JNI图像处理）、imagerecognition（形状识别+模板匹配+视频跟踪）、contentsearch（基于内容的图像检索CBIR）、qualityevaluation（图像质量评估PSNR/SSIM）、digitalwatermark（数字水印DCT/LSB）。项目技术栈为Kotlin + Java 11 + C++11 + OpenCV 4.x + AGP 9.2.1，compileSdk=37，minSdk=24，支持arm64-v8a和armeabi-v7a双架构。

项目已有的识别相关基础能力包括：Hu不变矩形状识别（5类形状分类）、NCC归一化互相关模板匹配（金字塔粗到精搜索）、视频模板跟踪（首帧取模板+邻域NCC搜索）、帧差运动检测（相邻帧差分二值化）。但尚未实现人脸检测、通用目标检测、Haar级联分类器、HOG行人检测、SVM分类器等高级识别算法。

1.3 术语与缩略语

术语	全称	说明
Haar	Haar-like Features	哈尔特征，基于矩形黑白区域亮度差的特征描述子
HOG	Histogram of Oriented Gradients	梯度方向直方图，用于物体检测的特征描述子
SVM	Support Vector Machine	支持向量机，二分类监督学习模型
CNN	Convolutional Neural Network	卷积神经网络，深度学习模型
LBP	Local Binary Pattern	局部二值模式，纹理特征描述子
NCC	Normalized Cross Correlation	归一化互相关，模板匹配相似度度量
MOG2	Mixture of Gaussians v2	混合高斯背景建模算法
KNN	K-Nearest Neighbors	K近邻背景减除算法
ROI	Region of Interest	感兴趣区域
BMA	Block Matching Algorithm	块匹配运动估计算法
MV	Motion Vector	运动矢量
NMS	Non-Maximum Suppression	非极大值抑制
PCA	Principal Component Analysis	主成分分析，用于特征脸方法
JNI	Java Native Interface	Java原生接口，用于调用C/C++代码

第二章 需求分析

2.1 功能需求

本模块需在现有imagerecognition模块基础上扩展，实现三大功能子系统：视频识别子系统、物体识别子系统、人脸识别子系统。各子系统功能需求如下：

2.1.1 视频识别子系统

功能ID	功能名称	功能描述	优先级
V-01	运动目标检测	基于背景建模（MOG2/KNN）和帧差法检测视频中的运动目标，输出前景掩码和外接矩形框	高
V-02	目标跟踪	基于NCC模板匹配和光流法实现对运动目标的连续跟踪，输出跟踪轨迹	高
V-03	运动估计	基于块匹配算法（BMA）计算相邻帧间的运动矢量场，可视化运动方向和大小	中
V-04	视频结构化分析	对视频进行镜头边界检测、关键帧提取、镜头聚类，输出视频结构化摘要	中
V-05	多目标检测与跟踪	在视频中同时检测和跟踪多个运动目标，分配唯一ID并绘制轨迹	中

2.1.2 物体识别子系统

功能ID	功能名称	功能描述	优先级
O-01	Haar特征目标检测	基于Haar级联分类器实现通用目标检测（人脸/眼睛/车身等），支持加载自定义级联模型	高
O-02	HOG行人检测	基于HOG特征描述子+SVM分类器实现行人检测，输出检测框和置信度	高
O-03	形状识别增强	在现有Hu不变矩形状识别基础上增加更多形状类别和分类精度	中
O-04	模板匹配增强	在现有NCC匹配基础上增加多尺度、多角度旋转匹配	中
O-05	特征点匹配	基于SIFT/ORB特征点提取与匹配实现物体识别	中
O-06	视频物体检测	对视频帧逐帧执行物体检测，输出每帧检测结果和统计摘要	高

2.1.3 人脸识别子系统

功能ID	功能名称	功能描述	优先级
F-01	人脸检测	基于Haar级联分类器实现静态图像人脸检测，支持多尺度检测和NMS去重	高
F-02	人脸关键点定位	检测眼睛、鼻子、嘴巴等面部关键点位置	中
F-03	人脸特征提取	基于LBP/PCA特征脸方法提取人脸特征向量	高
F-04	人脸识别	将检测到的人脸与数据库中已知人脸进行匹配，输出身份识别结果	高
F-05	视频人脸检测与跟踪	在视频流中实时检测人脸并跟踪，支持多脸跟踪	高

功能ID	功能名称	功能描述	优先级
F-06	人脸质量评估	评估检测到的人脸图像质量（清晰度、姿态、遮挡）	低

2.2 非功能需求

类别	需求描述	指标
性能-图像	单张图像人脸/物体检测延迟	< 500ms (480x480灰度图)
性能-视频	视频逐帧检测处理帧率	>= 10 fps (480p分辨率)
性能-跟踪	目标跟踪单帧处理延迟	< 50ms
内存	峰值内存占用	< 256MB
兼容性	Android最低版本	minSdk=24 (Android 7.0)
兼容性	ABI架构	arm64-v8a, armeabi-v7a
精度-人脸检测	正脸检测召回率	>= 90%
精度-跟踪	NCC跟踪命中率(无遮挡)	>= 80%
存储	Haar级联模型文件大小	< 5MB (打包在assets中)
存储	人脸特征数据库	本地存储，< 10MB

2.3 输入输出规格

模块的输入来源包括：(1) 项目assets目录下的静态图像 app/src/main/assets/IMG_20260821_190758.jpg；(2) 项目assets目录下的测试视频 app/src/main/assets/video.mp4；(3) 用户通过相机实时采集的预览帧（可选扩展）。

输出包括：(1) 标注了检测结果的Bitmap图像（检测框、关键点、跟踪轨迹叠加在原图上）；(2) 视频逐帧检测结果列表，每帧包含检测对象列表（类别、置信度、外接框坐标）；(3) 结构化分析报告（视频摘要、镜头列表、关键帧列表）；(4) 人脸特征数据库（特征向量+标签）。

第三章 实现原理详解

3.1 视频识别原理

3.1.1 运动估计与块匹配算法（BMA）

运动估计是视频压缩编码的核心技术（教材第5章5.3.3节），也是视频运动分析的基础。其基本原理是利用视频序列相邻帧间极强的时间相关性，通过估计运动物体的位移来减少时间域冗余。

块匹配算法（BMA）是最常用的运动估计方法。其核心假设是：同一子块内所有像素做相同平移运动。算法将当前帧分割为M x N的子块（如16x16），在上一帧的搜索区域内寻找最佳匹配块，匹配块与前块的相对位移即为运动矢量（Motion Vector, MV）。

■■■■■

SAD (Sum of Absolute Differences): $SAD = \sum |I(t, x, y) - I(t-1, x+dx, y+dy)|$

```
NCC (Normalized Cross Correlation):  $NCC = \frac{\text{Sum}(I1 \cdot I2)}{\sqrt{\text{Sum}(I1^2) \cdot \text{Sum}(I2^2)}}$   
MSE (Mean Squared Error):  $MSE = \frac{\text{Sum}(I1 - I2)^2}{M \cdot N}$ 
```

搜索策略：教材第5章介绍了三种搜索策略：(1) 全搜索 (Exhaustive Search) 精度最高但计算量大；(2) 三步搜索法 (3-Step Search)：步长依次减半，从最大搜索范围开始逐步缩小；(3) 分级搜索 (Multiscale Search)：先在低分辨率上全搜索（粗搜索），再以结果作为高分辨率细搜索的起始点。

H.264/AVC运动估计增强（教材第6章6.3节）：H.264引入了块大小可变的运动补偿（1个宏块可含1-16个运动矢量）、高精度亚像素运动估计（亮度1/4像素精度，色度1/8像素精度）、多参考帧运动补偿预测（最多5个参考帧）。这些技术可应用于本模块的多精度目标跟踪。

3.1.2 光流法

光流 (Optical Flow) 是图像中像素的运动模式，描述了相邻帧间像素的位移矢量场（教材第9章）。光流法计算相邻帧之间的光流场，可得到像素级精确的运动信息，但计算复杂度较高。

Horn-Schunck算法：基于亮度恒定假设和平滑约束的全局光流方法。亮度恒定方程： $I(x+u, y+v, t+1) = I(x, y, t)$ ，泰勒展开得： $I_x \cdot u + I_y \cdot v + I_t = 0$ ，其中 I_x, I_y 为空间梯度， I_t 为时间梯度， (u, v) 为光流矢量。

Lucas-Kanade算法：基于局部窗口的稀疏光流方法，假设窗口内像素具有相同运动，通过最小二乘法求解运动矢量。计算量小于全局方法，适合实时应用。OpenCV中的金字塔Lucas-Kanade算法（`cv::calcOpticalFlowPyrLK`）采用多分辨率金字塔提升大位移鲁棒性。

Farneback算法：基于多项式展开的稠密光流方法，OpenCV中通过`cv::calcOpticalFlowFarneback()`实现，可计算每个像素的完整运动矢量场，适合运动目标检测和可视化。

3.1.3 背景建模与前景提取

背景建模是运动目标检测的关键预处理步骤。基本思路是建立背景模型，将当前帧与背景模型比较，差异较大的像素判定为前景（运动目标）。

1. 帧差法（已有实现）：相邻帧差分二值化，统计变化像素占比与外接框。优点是计算简单快速，缺点是无法处理静止目标和背景渐变。公式： $D(x, y) = |I(t, x, y) - I(t-1, x, y)|$ ，若 $D > T$ 则为前景。

2. 混合高斯模型 (MOG2)：对每个像素建立K个高斯分布（通常 $K=3-5$ ），自适应更新背景模型。能够处理缓慢光照变化和周期性运动背景（如树叶摇摆）。像素值概率密度： $P(x) = \text{Sum}(w_i \cdot N(x, \mu_i, \sigma_i^2))$ 。OpenCV通过`cv::createBackgroundSubtractorMOG2()`实现。

3. KNN背景减除：对每个像素维护最近的N个样本值，当前像素与最近邻的距离超过阈值则判定为前景。OpenCV通过`cv::createBackgroundSubtractorKNN()`实现，对非周期性背景运动有更好适应性。

4. 形态学后处理：对前景掩码进行开运算去除噪声、闭运算填充空洞、腐蚀膨胀去除孤立点，提取连通域外接矩形作为目标区域。利用imageCVdeal模块已有的形态学算法实现。

3.1.4 目标跟踪算法

目标跟踪是在视频序列中持续定位特定目标的过程。本模块实现以下跟踪策略：

1. NCC模板跟踪（已有基础，需增强）：首帧中央取模板（48x48），后续帧在上一帧位置邻域（半径32像素）内用NCC搜索最佳匹配。增强方向：多尺度金字塔搜索、旋转不变性、丢失检测与重定位。

2. 光流跟踪：利用Lucas-Kanade金字塔光流法跟踪特征点，在首帧检测Shi-Tomasi角点（`cv::goodFeaturesToTrack`），后续帧用`cv::calcOpticalFlowPyrLK()`跟踪。优势：可跟踪多个稀疏特征点，对部分遮挡有鲁棒性。

3. Meanshift/Camshift跟踪：Meanshift基于颜色直方图反向投影的概率密度梯度上升搜索目标区域；Camshift在Meanshift基础上自适应调整窗口大小和旋转角度。适合跟踪颜色特征明显的目标。

4. 多目标跟踪：结合检测器（Haar/HOG）和跟踪器（NCC/光流），实现检测-跟踪融合框架。首帧检测初始化目标，后续帧用跟踪器维持，定期用检测器校正和发现新目标。使用匈牙利匹配或IoU关联管理目标ID。

3.1.5 视频结构化分析（教材第10章）

视频结构化是将视频从底层帧序列组织为高层语义结构的过程，分层结构为：视频 -> 场景(Scene) -> 镜头(Shot) -> 关键帧(Key Frame) -> 帧(Frame)。

镜头边界检测：计算相邻帧间距离（直方图差异、像素差异），采用双阈值检测法：帧间距离 $> TC$ 判定为镜头突变； $TG < \text{帧间距离} < TC$ 判定为镜头渐变。可利用contentsearch模块已有的ImageFeatures HSV颜色直方图计算帧间距离。

关键帧提取：基于颜色特征的方法--比较当前帧与上一个关键帧的颜色特征，变化超过阈值则作为新关键帧。也可采用帧平均法（取与镜头内所有帧平均值最接近的帧）。可复用contentsearch模块的ImageSearchEngine特征提取能力。

镜头聚类（场景检测）：基于关键帧特征将镜头聚集成场景，采用K-Means或层次聚类算法，反映高层语义。

3.2 物体识别原理

3.2.1 特征提取方法

特征提取是物体识别的核心环节（教材第11章11.1节），识别系统框架为：图像获取 -> 预处理 -> 特征提取 -> 分类决策。

1. Haar-like特征：由Paul Viola和Michael Jones于2001年提出（教材第11章提及），是基于矩形黑白区域亮度差的简单特征。每个特征由2-3个矩形组成，特征值 = 白色矩形像素和 - 黑色矩形像素和。利用积分图（Integral Image）可在 $O(1)$ 时间计算任意矩形区域的像素和。

Haar特征类型包括：边缘特征（2矩形）、线特征（3矩形）、对角线特征（4矩形）、中心特征等。通过AdaBoost算法从海量特征中筛选出最具分类能力的少量关键特征，构建级联分类器（Cascade Classifier）实现快速检测。

2. HOG特征（梯度方向直方图）：计算每个像素的梯度幅值和方向，将图像划分为小单元格（Cell，如 8×8 ），在每个Cell内统计梯度方向直方图（通常9个方向bin， $0-180^\circ$ ），将相邻Cell组成块（Block，如 2×2 Cells），块内归一化，最终将所有块的直方图串联为特征向量。

HOG特征对光照变化和局部几何形变有较强鲁棒性，配合SVM分类器广泛应用于行人检测。OpenCV通过cv::HOGDescriptor实现，内置预训练的行人检测模型。

3. SIFT特征（尺度不变特征变换）：通过DoG（Difference of Gaussian）金字塔检测尺度空间极值点，计算主方向（基于局部梯度方向直方图），生成128维特征描述子（ $4 \times 4 \times 8 = 128$ ）。具有尺度、旋转、光照不变性，适合特征点匹配识别。OpenCV 4.x通过cv::SIFT::create()和cv::SIFT::detectAndCompute()实现。

4. LBP特征（局部二值模式）：对每个像素，以其为中心与周围 3×3 邻域8个像素比较，大于等于中心值记1，否则记0，得到8位二进制编码（0-255）。统计LBP编码直方图作为纹理特征。具有光照不变性，计算简单，广泛应用于人脸识别。

3.2.2 分类器设计

1. 支持向量机 (SVM) (教材第11章11.2节) : SVM寻找最优超平面 $f(x) = w^T x + b$ 将不同类别分开, 使支持向量 (距超平面最近的样本) 到超平面的间隔 $\gamma = 2 / ||w||$ 最大。

```
min (1/2)||w||^2 s.t. y_i(w^T x_i + b) >= 1
max Sum(a_i) - (1/2)Sum_i Sum_j a_i a_j y_i y_j x_i^T x_j
K(x_i, x_j) = phi(x_i)^T phi(x_j)
RBF K(x,y) = exp(-||x-y||^2 / (2*sigma^2))
Sigmoid
```

经验风险最小化 (ERM) : 用训练样本平均损失最小化代替期望风险最小化。结构风险最小化 (SRM) : 在经验风险基础上加正则化项防止过拟合: $R_{\text{srn}} = R_{\text{emp}} + \lambda J(f)$ 。SVM等价于SRM的一种实现。

2. AdaBoost级联分类器: 用于Haar特征人脸/目标检测。级联结构由多个强分类器串联组成, 每个强分类器由多个弱分类器 (决策桩) 通过AdaBoost加权组合。前级使用少量特征快速排除大部分负样本, 后级使用更多特征精细判别。OpenCV通过`cv::CascadeClassifier`实现, 加载XML格式的训练模型。

3. 卷积神经网络CNN (教材第11章11.4节) : CNN基于三个核心概念: 局部感受野 (LRF, 神经元只连接局部区域)、权值共享 (同一卷积核所有位置使用相同权重, 大幅减少参数)、池化/下采样 (最大值池化或平均值池化, 减少数据量并增加平移不变性)。

CNN基本结构: 输入层 -> 卷积层 (特征提取) -> 池化层 (下采样) -> 全连接层 (分类) -> 输出层。卷积操作: $y_{ij} = \text{Sum}_m \text{Sum}_n (x_{\{i+m-1, j+n-1\}} * w_{\{mn\}})$ 。

教材以LeNet-5为例介绍了经典CNN架构: C1卷积层(6个5x5卷积核) -> S2下采样(6个14x14) -> C3卷积(16个5x5核) -> S4下采样 -> C5卷积(120个全连接) -> F6全连接(84个) -> 输出层(10个数字类别)。考虑到项目不使用深度学习框架, CNN部分作为理论参考, 实际实现以传统方法为主。

3.2.3 模板匹配 (已有基础, 需增强)

项目已有NccMatcher实现归一化互相关模板匹配, 采用粗到精金字塔搜索策略 (1/4分辨率全图搜索 -> 全分辨率局部精化)。NCC公式: $NCC = \text{Sum}(I1 * I2) / \sqrt{\text{Sum}(I1^2) * \text{Sum}(I2^2)}$, 对线性光照变化不变。

增强方向: (1) 多尺度匹配--在不同缩放因子下搜索, 适应目标尺寸变化; (2) 旋转匹配--在多个角度下搜索, 适应目标旋转; (3) 多模板匹配--支持多个模板同时匹配, 识别不同物体。

3.2.4 形状识别 (已有基础, 需增强)

项目已有ShapeRecognizer基于Hu不变矩实现5类形状 (圆/矩/三角/星/椭圆) 识别。Hu不变矩具有平移、旋转、尺度不变性, 7个矩经过对数变换后用于最近邻分类。增强方向: (1) 增加更多形状类别 (如六边形、十字、箭头等); (2) 使用SVM替代最近邻分类器提升精度; (3) 结合轮廓的傅里叶描述子增强形状区分能力。

3.3 人脸识别原理

3.3.1 Haar级联分类器人脸检测

Viola-Jones人脸检测框架 (2001年) 是第一个实现实时人脸检测的方法 (教材第11章提及), 由三个核心组件构成:

1. 积分图 (Integral Image) : 快速计算矩形区域像素和的辅助图像。积分图 $II(x,y) = \text{Sum}_{\{x' < x, y' < y\}} I(x', y')$, 任意矩形区域D的像素和只需4次积分图查表: $\text{Sum}(D) = II(\text{左上}) + II(\text{右下}) - II(\text{右上}) - II(\text{左下})$ 。这使得Haar特征计算复杂度从 $O(M*N)$ 降到 $O(1)$ 。

2. AdaBoost特征选择与分类器训练: 从海量Haar特征中筛选少量最具分类能力的特征构建弱分类器 (每个弱分类器对应一个特征), 通过AdaBoost算法迭代加权组合弱分类器为强分类器。每轮迭代增加被错分样本的权重, 减少正确分类样本的权重。

3. 级联结构 (Cascade)：多个强分类器串联，前级使用少量特征快速排除大部分非人脸区域（负样本），后级使用更多特征精细判别。通常级联有20-25级，前几级使用1-2个特征可排除约50%的负样本。检测时对图像多尺度滑动窗口扫描，每个尺度的每个窗口依次通过级联分类器。

OpenCV通过`cv::CascadeClassifier`实现，加载预训练的XML模型文件（如`haarcascade_frontalface_default.xml`）。关键方法：`detectMultiScale(image, objects, scaleFactor, minNeighbors, flags, minSize, maxSize)`。`scaleFactor`控制多尺度缩放比例（通常1.1-1.3），`minNeighbors`控制NMS去重的邻域重叠阈值。

3.3.2 特征脸方法 (Eigenface / PCA)

特征脸方法基于主成分分析 (PCA)，将高维人脸图像投影到低维特征子空间。步骤如下：

```
1. N MxN M*N
2. mu = (1/N) * Sum(x_i)
3. phi_i = x_i - mu
4. C = A * A^T A*Phi
5. C k U
6. omega = U^T * (x - mu) k
7.
```

3.3.3 LBP特征人脸识别

LBP（局部二值模式）是简单有效的纹理特征描述子，在人脸识别中应用广泛。基本步骤：

```
1. 3x3 LBP 8
2. m x n 7x7=49
3. LBP 256
4. 49*256=12544
5.
```

LBP+直方图方法对光照变化有较强鲁棒性，且计算简单无需训练。统一LBP（Uniform LBP）将256种模式映射为59种，进一步降维。

3.3.4 人脸识别完整流程

完整的人脸识别系统包括检测、对齐、特征提取、匹配四个阶段：

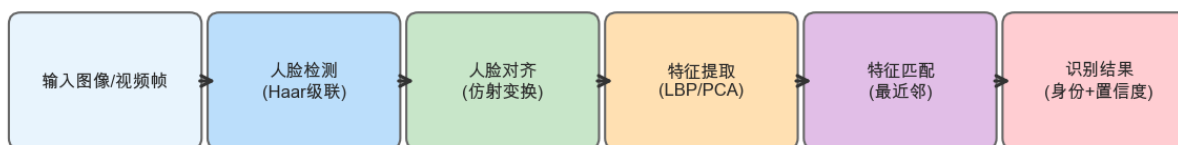


图 3-1: 人脸识别完整流程

1. 人脸检测：使用Haar级联分类器在图像中定位人脸区域，输出人脸外接矩形框。对视频帧逐帧执行检测，可结合跟踪减少重复检测开销。
2. 人脸对齐：利用眼睛关键点计算旋转角度，通过仿射变换将人脸归一化为正面朝向、固定尺寸（如100x100），消除姿态差异。
3. 特征提取：对对齐后的人脸提取LBP直方图特征或PCA特征脸特征，生成固定维度的特征向量。
4. 特征匹配：将提取的特征向量与数据库中已知人脸特征比对，使用卡方距离或欧氏距离度量，距离最小者且低于阈值则判定为匹配。未匹配则标记为未知人脸。

3.4 关键数学公式汇总

编号	公式	出处	说明
F-01	$SAD = \sum x_1 - x_2 $	Ch.5	块匹配绝对误差和
F-02	$NCC = \frac{\sum(I_1 * I_2)}{\sqrt{\sum(I_1^2) * \sum(I_2^2)}}$	已有代码	归一化互相关
F-03	$I_x * u + I_y * v + I_t = 0$	Ch.9	光流亮度恒定方程
F-04	$P(x) = \sum(w_i * N(\mu_i, \sigma_i^2))$	Ch.9	混合高斯背景模型
F-05	$f(x) = w^T * x + b$	Ch.11	SVM超平面
F-06	$\gamma = \frac{2}{ w }$	Ch.11	SVM分类间隔
F-07	$\min(1/2) w ^2 \text{ s.t. } y_i(w^T x_i + b) \geq 1$	Ch.11	SVM原始优化
F-08	$K(x, y) = \exp(- x - y ^2 / (2 * \sigma^2))$	Ch.11	RBF核函数
F-09	$R_{\text{srn}} = R_{\text{emp}} + \lambda * J(f)$	Ch.11	结构风险最小化
F-10	$II(x, y) = \sum(I(x', y'))$	Ch.11	积分图定义
F-11	$y_{ij} = \sum_m \sum_n (x * w_{mn})$	Ch.11	卷积操作
F-12	$f(x) = \max(0, x)$	Ch.11	ReLU激活函数
F-13	$\omega = U^T * (x - \mu)$	PCA	特征脸投影
F-14	$LBP = \sum(2^i * s(p_i - p_c))$	纹理	LBP编码

第四章 代码实现架构

4.1 模块整体架构

在现有imagerecognition模块基础上扩展，新增三个子包：video（视频识别）、object（物体识别）、face（人脸识别）。复用imageCVdeal的OpenCV JNI能力和contentsearch的特征提取能力。

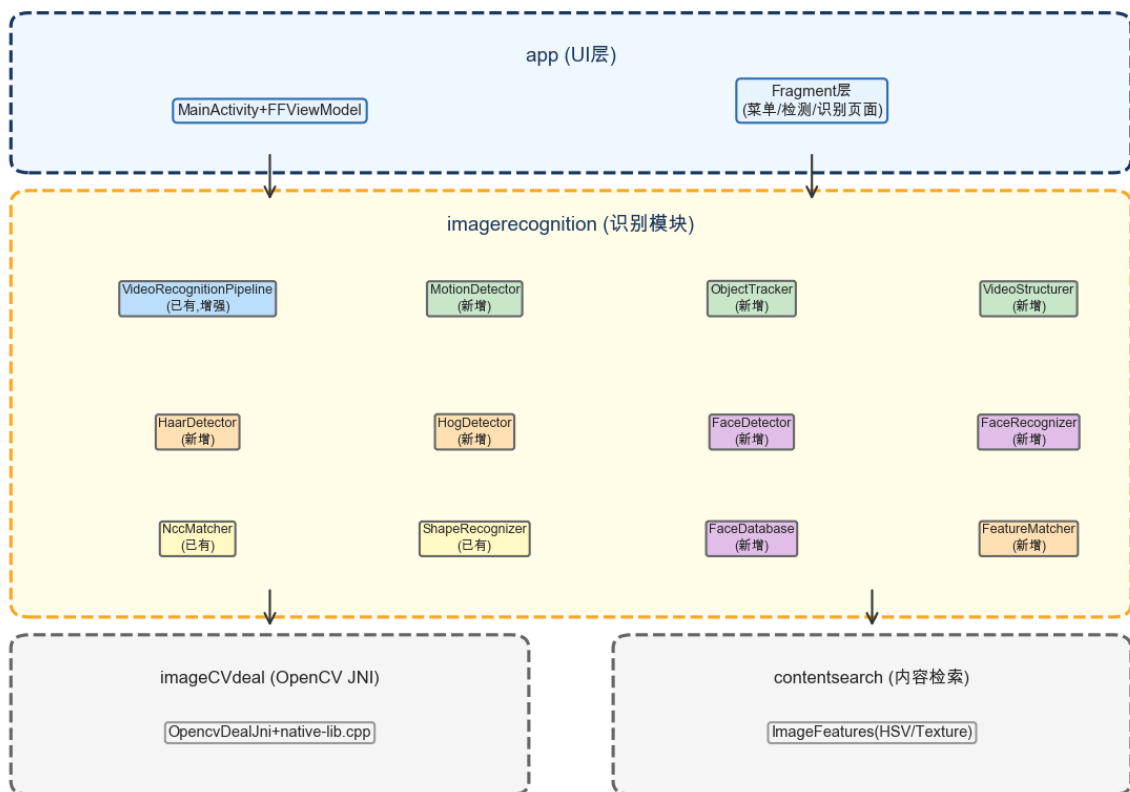


图 4-1: 模块整体架构图

4.2 新增包与文件结构

在imagerecognition模块中新增以下文件结构：

```
imagerecognition/src/main/java/com/wangyao/imagerecognition/ core/ NccMatcher.kt (■■,■■■■■■/■■)
ShapeRecognizer.kt (■■,■■■■■■) MotionDetector.kt (■■: MOG2/KNN/■■■■■■) ObjectTracker.kt (■■:
NCC/■■/Meanshift■■) VideoStructurer.kt (■■: ■■■■/■■■/■■■■) object/ HaarDetector.kt (■■:
Haar■■■■■■) HogDetector.kt (■■: HOG+SVM■■■■) FeatureMatcher.kt (■■: SIFT/ORB■■■■■■)
DetectionResult.kt (■■: ■■■■■■■■) face/ FaceDetector.kt (■■: Haar■■■■+■■■■) FaceRecognizer.kt (■■:
LBP/PCA■■■■■■) FaceDatabase.kt (■■: ■■■■■■■■■■) FaceAlignment.kt (■■: ■■■■■) video/
VideoRecognitionPipeline.kt (■■,■■■■■■■■■■■■) VideoObjectDetector.kt (■■: ■■■■■■■■■■)
MultiObjectTracker.kt (■■: ■■■■■■■■■■)
```

4.3 核心类设计

4.3.1 MotionDetector (运动检测器)

职责：基于背景建模和帧差法检测视频中的运动目标，输出前景掩码和目标外接框。

```
// MotionDetector.kt package com.wangyao.imagerecognition.core import org.opencv.core.* import
org.opencv.imgproc.Imgproc import org.opencv.video.Video import org.opencv.video.BackgroundSubtractor
class MotionDetector( private val method: Method = Method.MOG2, private val history: Int = 500,
private val varThreshold: Double = 16.0 ) { enum class Method { FRAME_DIFF, MOG2, KNN } private var
subtractor: BackgroundSubtractor? = null private var prevGray: Mat? = null init { when (method) {
Method.MOG2 -> subtractor = Video.createBackgroundSubtractorMOG2( history, varThreshold, true)
Method.KNN -> subtractor = Video.createBackgroundSubtractorKNN( history, varThreshold, true)
Method.FRAME_DIFF -> { } } } data class Detection( val motionRatio: Double, val boxes: List, val
foregroundMask: ByteArray ) fun detect(gray: Mat, width: Int, height: Int): Detection { val mask =
Mat() when (method) { Method.FRAME_DIFF -> { if (prevGray != null) { Core.absdiff(gray, prevGray!!,
```

```
mask) Imgproc.threshold(mask, mask, 25.0, 255.0, Imgproc.THRESH_BINARY) } else { mask =
Mat.zeros(gray.size(), gray.type()) } prevGray = gray.clone() } Method.MOG2, Method.KNN -> {
subtractor?.apply(gray, mask) } } // ■■■■■■ val kernel = Imgproc.getStructuringElement(
Imgproc.MORPH_ELLIPSE, Size(5.0, 5.0)) Imgproc.morphologyEx(mask, mask, Imgproc.MORPH_OPEN, kernel)
Imgproc.morphologyEx(mask, mask, Imgproc.MORPH_CLOSE, kernel) // ■■■■■■ val contours = ArrayList()
Imgproc.findContours(mask, contours, Mat(), Imgproc.RETR_EXTERNAL, Imgproc.CHAIN_APPROX_SIMPLE) val
minArea = width * height * 0.005 val boxes = contours .map { Imgproc.boundingRect(it) } .filter {
it.area() > minArea } .toList() val motionPixels = Core.countNonZero(mask) val motionRatio =
motionPixels.toDouble() / (width * height) return Detection(motionRatio, boxes, mask.toArray()) } fun
release() { subtractor?.clear() prevGray?.release() } }
```

4.3.2 ObjectTracker（目标跟踪器）

职责：在视频序列中持续定位特定目标，支持NCC模板跟踪和光流跟踪两种模式。

```
// ObjectTracker.kt package com.wangyao.imagerecognition.core import org.opencv.core.* import
org.opencv.imgproc.Imgproc import org.opencv.video.Video class ObjectTracker( private val method:
TrackMethod = TrackMethod.NCC, private val searchRadius: Int = 32, private val templateSize: Int =
48, private val lostThreshold: Double = 0.5 ) { enum class TrackMethod { NCC, OPTICAL_FLOW } data
class TrackResult( val x: Double, val y: Double, val score: Double, val isLost: Boolean, val
trajectory: List ) private var template: Mat? = null private var prevGray: Mat? = null private var
prevPoints: MatOfPoint2f? = null private var centerX: Double = 0.0 private var centerY: Double = 0.0
private val trajectory = mutableListOf() private var lostCount: Int = 0 fun init(gray: Mat, startX:
Int, startY: Int) { centerX = startX.toDouble() centerY = startY.toDouble() when (method) {
TrackMethod.NCC -> { val half = templateSize / 2 template = Mat(gray, Rect( startX - half, startY -
half, templateSize, templateSize)).clone() } TrackMethod.OPTICAL_FLOW -> { val corners =
MatOfPoint2f() Imgproc.goodFeaturesToTrack(gray, corners, 50, 0.01, 10.0) prevPoints = corners
prevGray = gray.clone() } } trajectory.add(Point(centerX, centerY)) } fun track(gray: Mat):
TrackResult { return when (method) { TrackMethod.NCC -> trackNCC(gray) TrackMethod.OPTICAL_FLOW ->
trackOpticalFlow(gray) } } private fun trackNCC(gray: Mat): TrackResult { val tpl = template ?:
return TrackResult( centerX, centerY, 0.0, true, trajectory) val sx = (centerX -
searchRadius).toInt().coerceAtLeast(0) val sy = (centerY - searchRadius).toInt().coerceAtLeast(0) val
sw = minOf(searchRadius * 2 + templateSize, gray.cols() - sx) val sh = minOf(searchRadius * 2 +
templateSize, gray.rows() - sy) if (sw <= 0 || sh <= 0) return TrackResult(centerX, centerY, 0.0,
true, trajectory) val searchRegion = Mat(gray, Rect(sx, sy, sw, sh)) val result = Mat()
Imgproc.matchTemplate(searchRegion, tpl, result, Imgproc.TM_CCOEFF_NORMED) val minMax =
Core.minMaxLoc(result) val score = minMax.maxVal val maxLoc = minMax.maxLoc if (score >
lostThreshold) { centerX = sx + maxLoc.x + templateSize / 2.0 centerY = sy + maxLoc.y + templateSize /
2.0 lostCount = 0 } else { lostCount++ } trajectory.add(Point(centerX, centerY)) return
TrackResult(centerX, centerY, score, score < lostThreshold, trajectory) } private fun
trackOpticalFlow(gray: Mat): TrackResult { val prev = prevGray ?: return TrackResult( centerX,
centerY, 0.0, true, trajectory) val prevPts = prevPoints ?: return TrackResult( centerX, centerY,
0.0, true, trajectory) val nextPts = MatOfPoint2f() val status = MatOfByte() val err = MatOfFloat()
Video.calcOpticalFlowPyrLK(prev, gray, prevPts, nextPts, status, err) val statusArr =
status.toArray() val nextArr = nextPts.toArray() var sumX = 0.0; var sumY = 0.0; var count = 0 for (i
in statusArr.indices) { if (statusArr[i] == 1.toByte()) { sumX += nextArr[i].x; sumY += nextArr[i].y;
count++ } } if (count > 0) { centerX = sumX / count; centerY = sumY / count lostCount = 0; prevPoints
= nextPts; prevGray = gray.clone() } else { lostCount++ } trajectory.add(Point(centerX, centerY)) val
score = if (count > 0) count.toDouble() / statusArr.size else 0.0 return TrackResult(centerX,
centerY, score, count == 0, trajectory) } }
```

4.3.3 HaarDetector（Haar级联检测器）

职责：基于Haar级联分类器实现通用目标检测（人脸/眼睛/车身等），支持加载OpenCV预训练XML模型和自定义模型。

```
// HaarDetector.kt package com.wangyao.imagerecognition.object import org.opencv.core.* import
org.opencv.objdetect.CascadeClassifier class HaarDetector( private val cascadePath: String, private
val scaleFactor: Double = 1.1, private val minNeighbors: Int = 3, private val minSize: Size =
Size(30.0, 30.0), private val maxSize: Size = Size() ) { private val cascade: CascadeClassifier =
CascadeClassifier() data class Detection( val x: Double, val y: Double, val width: Double, val
height: Double, val confidence: Int ) init { if (!cascade.load(cascadePath)) throw
IllegalArgumentException("Failed to load: $cascadePath") } fun detect(gray: Mat): List { val objects
= MatOfRect() cascade.detectMultiScale(gray, objects, scaleFactor, minNeighbors, 0, minSize, maxSize)
return objects.toList().map { rect -> Detection(rect.x.toDouble(), rect.y.toDouble(),
```

```
rect.width.toDouble(), rect.height.toDouble(), minNeighbors) } } fun detectNested(gray: Mat,
nestedDetector: HaarDetector, roiPadding: Double = 0.1 ): Map< val outer = detect(gray) val result
= mutableMapOf<>() for (det in outer) { val padW = det.width * roiPadding val padH = det.height *
roiPadding val roi = Rect( maxOf(0, (det.x + padW * 0.5).toInt()), maxOf(0, (det.y + padH *
0.5).toInt()), minOf(gray.cols(), (det.width - padW).toInt()), minOf(gray.rows(), (det.height - padH
* 0.5).toInt())) if (roi.width > 0 && roi.height > 0) { val roiMat = Mat(gray, roi) val nested =
nestedDetector.detect(roiMat) result[det] = nested.map { it.copy(x = it.x + roi.x, y = it.y + roi.y)
} } } return result } }
```

4.3.4 HogDetector (HOG行人检测器)

职责：基于HOG特征描述子和SVM分类器实现行人检测。使用OpenCV内置的预训练HOG行人检测模型。

```
// HogDetector.kt package com.wangyao.imagerecognition.object import org.opencv.core.* import
org.opencv.objdetect.HOGDescriptor class HogDetector( private var hitThreshold: Double = 0.0, private
var winStride: Size = Size(8.0, 8.0), private var padding: Size = Size(4.0, 4.0), private var scale:
Double = 1.05 ) { private val hog: HOGDescriptor = HOGDescriptor() data class Detection( val x:
Double, val y: Double, val width: Double, val height: Double, val score: Double ) init {
hog.setSVMDetector(HOGDescriptor.getDefaultPeopleDetector()) } fun detect(gray: Mat): List { val
found = MatOfRect() val weights = MatOfDouble() hog.detectMultiScale(gray, found, weights,
hitThreshold, winStride, padding, scale) val rects = found.toList() val scores = weights.toList()
return rects.mapIndexed { i, rect -> Detection(rect.x.toDouble(), rect.y.toDouble(),
rect.width.toDouble(), rect.height.toDouble(), if (i < scores.size) scores[i] else 0.0)
}.sortedByDescending { it.score } }
```

4.3.5 FaceDetector & FaceRecognizer (人脸检测与识别)

FaceDetector职责：基于Haar级联分类器检测人脸位置，支持眼睛关键点定位。FaceRecognizer职责：基于LBP直方图特征和PCA特征脸方法实现人脸身份识别。

```
// FaceDetector.kt package com.wangyao.imagerecognition.face import org.opencv.core.* import
org.opencv.objdetect.CascadeClassifier class FaceDetector( private val faceCascadePath: String,
private val eyeCascadePath: String? = null ) { private val faceCascade = CascadeClassifier() private
val eyeCascade: CascadeClassifier? = eyeCascadePath?.let { CascadeClassifier().also { c -> c.load(it)
} } data class FaceDetection( val face: Rect, val eyes: List, val confidence: Double ) init {
require(faceCascade.load(faceCascadePath)) { "Failed to load face cascade: $faceCascadePath" } } fun
detect(gray: Mat): List { val faces = MatOfRect() faceCascade.detectMultiScale(gray, faces, 1.3, 5,
0, Size(30.0, 30.0), Size()) return faces.toList().map { faceRect -> val eyes = if (eyeCascade !=
null) { val roi = Mat(gray, faceRect) val eyeRects = MatOfRect() eyeCascade.detectMultiScale(roi,
eyeRects, 1.1, 3, 0) eyeRects.toList().map { Rect(it.x + faceRect.x, it.y + faceRect.y, it.width,
it.height) } } else emptyList() FaceDetection(faceRect, eyes, 1.0) } } } // FaceRecognizer.kt package
com.wangyao.imagerecognition.face import org.opencv.core.* import org.opencv.imgproc.* class
FaceRecognizer( private val method: Method = Method.LBP, private val faceSize: Int = 100, private val
gridX: Int = 7, private val gridY: Int = 7, private val numComponents: Int = 80 ) { enum class Method
{ LBP, PCA_EIGENFACE } data class RecognitionResult( val label: String, val confidence: Double, val
distance: Double, val isUnknown: Boolean ) private val features = mutableListOf<>() fun
extractFeature(alignedFace: Mat): FloatArray { return when (method) { Method.LBP ->
extractLBP(alignedFace) Method.PCA_EIGENFACE -> extractPCA(alignedFace) } } private fun
extractLBP(face: Mat): FloatArray { val lbp = Mat(face.size(), CvType.CV_8UC1) for (i in 1 until
face.rows() - 1) { for (j in 1 until face.cols() - 1) { val center = face.get(i, j)[0] var code = 0
val neighbors = arrayOf( face.get(i-1,j-1)[0], face.get(i-1,j)[0], face.get(i-1,j+1)[0],
face.get(i,j+1)[0], face.get(i+1,j+1)[0], face.get(i+1,j)[0], face.get(i+1,j-1)[0],
face.get(i,j-1)[0]) for (k in neighbors.indices) if (neighbors[k] >= center) code = code or (1 shl k)
lbp.put(i, j, code.toByte()) } } val cellW = face.cols() / gridX val cellH = face.rows() / gridY val
hist = FloatArray(gridX * gridY * 59) for (gy in 0 until gridY) { for (gx in 0 until gridX) { val roi
= Mat(lbp, Rect( gx*cellW, gy*cellH, cellW, cellH)) val histRoi = FloatArray(59) for (i in 0 until
roi.rows()) { for (j in 0 until roi.cols()) { val v = roi.get(i,j)[0].toInt() and 0xFF val u =
uniformMap(v) if (u < 59) histRoi[u]++ } } val norm = Math.sqrt( histRoi.sumOf { it.toDouble()*it
}).toFloat() if (norm > 0) for (k in histRoi.indices) histRoi[k] /= norm val offset = (gy * gridX +
gx) * 59 System.arraycopy(histRoi, 0, hist, offset, 59) } } return hist } private val uniformMap =
IntArray(256) { i -> if (countTransitions(i) <= 2) i else 58 } private fun countTransitions(v: Int):
Int { var count = 0 for (i in 0 until 8) { val b1 = (v shr i) and 1 val b2 = (v shr ((i+1)%8)) and 1
if (b1 != b2) count++ } return count } private fun extractPCA(face: Mat): FloatArray { val resized =
Mat() Imgproc.resize(face, resized, Size(faceSize.toDouble(), faceSize.toDouble())) val vector =
FloatArray(resized.rows()*resized.cols()) val data = ByteArray(vector.size) resized.get(0, 0, data)
```

```

for (i in data.indices) vector[i] = (data[i].toInt() and 0xFF).toFloat() return vector } fun
enroll(label: String, faceMat: Mat) { features.add(label to extractFeature(faceMat)) } fun
recognize(alignedFace: Mat): RecognitionResult { val query = extractFeature(alignedFace) var
bestLabel = "Unknown" var bestDist = Double.MAX_VALUE for ((label, feat) in features) { val d =
chiSquareDistance(query, feat) if (d < bestDist) { bestDist = d; bestLabel = label } } val threshold =
method.threshold val isUnknown = bestDist > threshold val confidence = maxOf(0.0, 1.0 - bestDist /
threshold) return RecognitionResult( if (isUnknown) "Unknown" else bestLabel, confidence, bestDist,
isUnknown) } private fun chiSquareDistance(a: FloatArray, b: FloatArray): Double { var sum = 0.0 for
(i in a.indices) { if (a[i] + b[i] > 0) { val d = a[i] - b[i] sum += d * d / (a[i] + b[i]) } } return
sum } companion object { val Method.threshold: Double get() = when (this) { Method.LBP -> 100.0
Method.PCA_EIGENFACE -> 5000.0 } } }

```

4.3.6 FaceDatabase（人脸特征数据库）

职责：管理人脸特征数据库的持久化存储，支持注册新用户、删除用户、查询所有用户。使用SharedPreferences存储元数据，内部存储目录存储特征向量。

```

// FaceDatabase.kt package com.wangyao.imagerecognition.face import android.content.Context import
java.io.* class FaceDatabase(private val context: Context) { private val prefs =
context.getSharedPreferences( "face_db", Context.MODE_PRIVATE) private val dir =
File(context.filesDir, "face_features").apply { if (!exists()) mkdirs() } data class FaceEntry( val
id: String, val label: String, val featurePath: String, val createdAt: Long ) fun enroll(label:
String, feature: FloatArray): String { val id = "${label}_${System.currentTimeMillis()}" val file =
File(dir, "$id.dat") ObjectOutputStream(FileOutputStream(file)).use { oos -> oos.writeObject(feature)
} prefs.edit() .putString("entry_$id", label) .putString("path_$id", file.absolutePath)
.putLong("time_$id", System.currentTimeMillis()) .apply() return id } fun listAll(): List { return
prefs.all.keys .filter { it.startsWith("entry_") } .map { key -> val id = key.removePrefix("entry_")
FaceEntry(id, prefs.getString(key, "") ?: "", prefs.getString("path_$id", "") ?: "",
prefs.getLong("time_$id", 0)) } } fun loadFeature(id: String): FloatArray? { val path =
prefs.getString("path_$id", null) ?: return null val file = File(path) if (!file.exists()) return
null return ObjectInputStream(FileInputStream(file)).use { ois -> ois.readObject() as FloatArray } }
fun delete(id: String) { prefs.getString("path_$id", null)?.let { File(it).delete() } prefs.edit()
.remove("entry_$id").remove("path_$id").remove("time_$id").apply() } fun count(): Int =
prefs.all.keys .count { it.startsWith("entry_") } }

```

4.3.7 VideoStructurer（视频结构化分析器）

职责：对视频进行镜头边界检测、关键帧提取、镜头聚类。复用contentsearch模块的ImageFeatures。

```

// VideoStructurer.kt package com.wangyao.imagerecognition.core import
com.wangyao.contentsearch.core.ImageFeatures data class Shot( val startFrame: Int, val endFrame: Int,
val keyFrameIndex: Int, val type: ShotType ) enum class ShotType { CUT, GRADUAL, SCENE } data class
VideoSummary( val totalFrames: Int, val fps: Double, val duration: Double, val shots: List, val
keyFrames: List, val scenes: List ) class VideoStructurer( private val cutThreshold: Double = 0.5,
private val gradualThreshold: Double = 0.3, private val keyFrameThreshold: Double = 0.15 ) { fun
analyze(videoPath: String, onProgress: (Int, Int) -> Unit ): VideoSummary { val shots =
mutableListOf() val keyFrames = mutableListOf() var prevHist: FloatArray? = null var prevKfHist:
FloatArray? = null var shotStart = 0 var gradualStart = -1
VideoRecognitionPipeline.decodeOnly(videoPath, onFrame = { luma, w, h, idx -> val hist =
ImageFeatures.colorHistogramFromLuma(luma) if (prevHist != null) { val dist =
histogramDistance(prevHist!!, hist) when { dist > cutThreshold -> { shots.add(Shot(shotStart, idx-1,
(shotStart+idx)/2, ShotType.CUT)) keyFrames.add(idx) shotStart = idx gradualStart = -1 } dist >
gradualThreshold -> { if (gradualStart < 0) gradualStart = idx } else -> { if (gradualStart >= 0) {
shots.add(Shot(gradualStart, idx, (gradualStart+idx)/2, ShotType.GRADUAL)) keyFrames.add(idx)
shotStart = idx gradualStart = -1 } if (keyFrames.isNotEmpty() && prevKfHist != null) { val kfDist =
histogramDistance( prevKfHist!!, hist) if (kfDist > keyFrameThreshold) { keyFrames.add(idx)
prevKfHist = hist } } else { keyFrames.add(idx) prevKfHist = hist } } } } else { keyFrames.add(idx)
prevKfHist = hist } prevHist = hist onProgress(idx, -1) } ) val scenes = clusterShots(shots,
keyFrames) return VideoSummary( totalFrames = shots.lastOrNull()?.endFrame ?: 0, fps = 30.0, duration
= (shots.lastOrNull()?.endFrame ?: 0) / 30.0, shots = shots, keyFrames = keyFrames, scenes = scenes)
} private fun histogramDistance(a: FloatArray, b: FloatArray): Double { var sum = 0.0 for (i in
a.indices) { if (a[i] + b[i] > 0) { val d = a[i] - b[i] sum += d * d / (a[i] + b[i]) } } return sum }
private fun clusterShots(shots: List, keyFrames: List): List { if (shots.isEmpty()) return
emptyList() return listOf(shots.indices.toList()) } }

```


4.4 JNI/OpenCV集成架构

本模块需要在imageCVdeal模块的CMakeLists.txt和native-lib.cpp中新增OpenCV识别相关函数。imageCVdeal已配置OpenCV 4.x动态库（libopencv_java4.so），头文件包含opencv2/core、imgproc、objdetect、video、ml、features2d等模块。

需要新增的OpenCV头文件依赖：

```
#include <opencv2/objdetect.hpp> // CascadeClassifier, HOGDescriptor
#include <opencv2/video.hpp> // BackgroundSubtractor, calcOpticalFlow
#include <opencv2/video/tracking.hpp> // KalmanFilter, goodFeaturesToTrack
#include <opencv2/features2d.hpp> // SIFT, ORB, FeatureMatcher
#include <opencv2/ml.hpp> // SVM, KNearest
```

OpencvDealJni.kt 新增接口：

```
// OpencvDealJni.kt
object OpencvDealJni {
    init {
        System.loadLibrary("opencv_java4")
        System.loadLibrary("opencvdeal_native")
    }
    // -- external fun nMotionDetectMOG2(gray: ByteArray, w: Int, h: Int, history: Int, varThreshold: Double): ByteArray
    external fun nMotionDetectKNN(gray: ByteArray, w: Int, h: Int, history: Int, distThreshold: Double): ByteArray
    // -- external fun nOpticalFlowLK(prevGray: ByteArray, curGray: ByteArray, w: Int, h: Int, prevPoints: FloatArray): FloatArray
    external fun nOpticalFlowFarneback(prevGray: ByteArray, curGray: ByteArray, w: Int, h: Int): FloatArray
    // -- Haar
    external fun nHaarDetect(gray: ByteArray, w: Int, h: Int, cascadePath: String, scaleFactor: Double, minNeighbors: Int, minWidth: Int, minHeight: Int): FloatArray
    // -- HOG
    external fun nHogDetect(gray: ByteArray, w: Int, h: Int, hitThreshold: Double, winStride: Int, padding: Int, scale: Double): FloatArray
    // -- external fun nSiftDetectAndCompute(gray: ByteArray, w: Int, h: Int): SiftResult
    external fun nFeatureMatch(desc1: FloatArray, desc2: FloatArray, method: Int): IntArray
    // -- external fun nExtractLBP(gray: ByteArray, w: Int, h: Int, gridX: Int, gridY: Int): FloatArray
    external fun nComputePCA(faces: Array, w: Int, h: Int, numComponents: Int): PCAResult
    // -- Meanshift/Camshift
    external fun nMeanshift(backProj: ByteArray, w: Int, h: Int, window: IntArray): IntArray
    external fun nCamshift(backProj: ByteArray, w: Int, h: Int, window: IntArray): CamshiftResult
}
```

4.5 视频处理管线设计

视频处理管线沿用项目已有的MediaExtractor + MediaCodec解码架构，提取YUV_420_888的Y亮度平面后送入识别算法。对于需要彩色信息的算法（如HSV颜色直方图），同时提取UV平面。

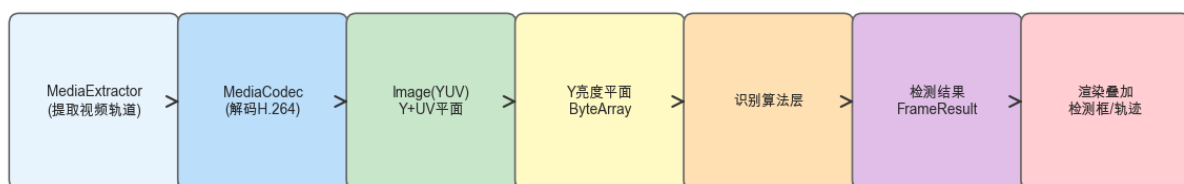


图 4-2: 视频处理管线流程

4.6 UI架构设计

UI层在app模块中新增Fragment和路由配置。沿用项目已有的FFViewModel + MainActivity路由架构。

```
// Fragment
// 1. Fragment (RecognitionMenuFragment.kt - RecognitionMenuFragment.kt // 2.
// VideoMotionDetectFragment.kt // VideoTrackingFragment.kt //
VideoStructureFragment.kt // 3. ObjectDetectFragment.kt // Haar/HOG
ShapeRecognitionFragment.kt // (FeatureMatchFragment.kt // SIFT/ORB
// 4.
FaceDetectFragment.kt // FaceRecognizeFragment.kt //
FaceVideoFragment.kt // (FFViewModel.kt) fun navigateToRecognition(target:
RecognitionTarget) { val fragment = when (target) { RecognitionTarget.MENU ->
RecognitionMenuFragment() RecognitionTarget.VIDEO_MOTION -> VideoMotionDetectFragment() }
```

```
RecognitionTarget.VIDEO_TRACKING -> VideoTrackingFragment() RecognitionTarget.VIDEO_STRUCTURE ->
VideoStructureFragment() RecognitionTarget.OBJECT_DETECT -> ObjectDetectFragment()
RecognitionTarget.FACE_DETECT -> FaceDetectFragment() RecognitionTarget.FACE_RECOGNIZE ->
FaceRecognizeFragment() RecognitionTarget.FACE_VIDEO -> FaceVideoFragment() }
switchFragment.postValue(fragment) }
```

4.7 数据流设计

模块内部数据流遵循统一的输入->处理->输出模式：

图像检测流程：Bitmap/ByteArray -> JNI灰度转换 -> OpenCV Mat ->

HaarDetector/HogDetector/FaceDetector -> Detection列表 -> 叠加渲染到Canvas -> 显示

视频识别流程：MediaExtractor -> MediaCodec -> YUV Y平面 -> MotionDetector/ObjectTracker -> FrameResult -> 逐帧叠加渲染 -> 输出标注视频

人脸识别流程：人脸检测 -> 人脸对齐(仿射变换) -> FaceRecognizer.extractFeature -> FaceDatabase.loadAll -> 最近邻匹配 -> RecognitionResult

第五章 核心算法实现细节

5.1 运动检测算法实现细节

运动检测支持三种方法：帧差法、MOG2混合高斯、KNN背景减除。

帧差法实现要点：(1) 相邻帧灰度差分 $D = |I(t) - I(t-1)|$ ；(2) 阈值二值化 $B = D > T ? 255 : 0$ ，T默认25；(3) 形态学开运算去噪、闭运算填洞；(4) findContours提取连通域外接矩形；(5) 面积过滤去除小噪声区域。

MOG2实现要点：(1) 初始化BackgroundSubtractorMOG2(history=500, varThreshold=16, detectShadows=true)；(2) 逐帧apply更新背景模型并输出前景掩码；(3) 阴影像素值为127，可选保留或删除；(4) 形态学后处理同帧差法。

5.2 目标跟踪算法实现细节

NCC模板跟踪（已有基础增强）：(1) 初始化：首帧在指定位置提取48x48灰度模板；(2) 搜索：在上一帧位置周围searchRadius=32像素范围内用TM_CCOEFF_NORMED匹配；(3) 更新：取NCC最大值位置作为新目标位置，若score > lostThreshold=0.5则跟踪成功；(4) 丢失处理：连续lostCount超过阈值则进入重定位模式；(5) 增强多尺度：在3个缩放因子(0.8, 1.0, 1.2)下搜索，适应目标尺寸变化。

光流跟踪实现要点：(1) goodFeaturesToTrack在ROI内检测最多50个Shi-Tomasi角点；(2) calcOpticalFlowPyrLK计算前后帧特征点对应关系，金字塔层数默认3；(3) 统计成功跟踪的特征点中心作为目标位置；(4) 剔除外点（距离中位数超过2倍MAD的点）；(5) 定期补充新特征点防止跟踪退化。

5.3 Haar级联检测算法实现细节

核心流程：(1) 加载XML级联模型文件（从assets复制到内部存储）；(2) 多尺度滑动窗口扫描：从minSize到maxSize，每次乘scaleFactor(1.1-1.3)；(3) 每个窗口通过级联分类器，前几级快速排除大部分负样本；(4) NMS去重：对重叠率超过0.3的检测框保留置信度最高的；(5) 嵌套检测：在人脸区域内检测眼睛/鼻子等子部件。

NMS（非极大值抑制）实现：(1) 按置信度降序排序所有检测框；(2) 取最高分框加入结果列表，移除与其IoU > 阈值(0.3)的所有框；(3) 重复直到候选框为空。

5.4 HOG行人检测算法实现细节

(1) 使用HOGDescriptor默认参数：winSize=(64,128), blockSize=(16,16), cellSize=(8,8), bin=9, 总特征维度=3780；(2) 加载getDefaultPeopleDetector()预训练SVM权重；(3) detectMultiScale多尺度检测，winStride=(8,8)控制滑动步长，scale=1.05；(4) NMS去重同Haar检测；(5) 输出检测框和置信度权重。

5.5 人脸识别算法实现细节

LBP特征提取流程：(1) 人脸对齐：利用眼睛位置计算旋转角度，仿射变换归一化为100x100正面人脸；(2) 逐像素计算3x3邻域LBP编码（8位二进制，256种模式）；(3) 映射为Uniform LBP（最多2次0-1跳变的模式，共58种+1种其他=59类）；(4) 7x7=49个子区域分别统计59维直方图；(5) L2归一化每个子区域直方图；(6) 串联为49*59=2891维特征向量。

人脸匹配流程：(1) 计算查询特征与数据库所有特征之间的卡方距离；(2) 取距离最小者作为候选；(3) 若距离 < 阈值(100.0)则判定为已知身份，否则标记为未知；(4) 置信度 = 1 - distance/threshold。

5.6 视频结构化分析实现细节

镜头边界检测：(1) 逐帧提取HSV颜色直方图（复用contentsearch.ImageFeatures）；(2) 计算相邻帧直方图卡方距离；(3) 双阈值判定：距离 > cutThreshold(0.5)为突变，gradualThreshold(0.3) < 距离 < cutThreshold为渐变；(4) 同时计算帧间距离D(k,k+1)和跨帧距离D(k,k+L)提高检测精度。

关键帧提取：(1) 基于颜色特征法：比较当前帧与上一个关键帧的直方图距离；(2) 距离 > keyFrameThreshold(0.15)则选为新关键帧；(3) 每个镜头至少包含首帧和末帧作为关键帧。

第六章 接口定义

6.1 JNI接口规范

接口名	参数	返回值	功能
nMotionDetectMOG2	gray,w,h,history,varThreshold	ByteArray(前景掩码)	MOG2背景减除
nMotionDetectKNN	gray,w,h,history,distThreshold	ByteArray(前景掩码)	KNN背景减除
nOpticalFlowLK	prevGray,curGray,w,h,prevPoints	FloatArray(点+状态)	LK金字塔光流
nOpticalFlowFarneback	prevGray,curGray,w,h	FloatArray(光流场)	Farneback稠密光流
nHaarDetect	gray,w,h,cascadePath,scale,neighbors,minW,minH	FloatArray(检测框列表)	Haar级联检测
nHogDetect	gray,w,h,hitThreshold.winStride,padding,scale	FloatArray(检测框列表)	HOG行人检测

接口名	参数	返回值	功能
nSiftDetectAndCompute	gray,w,h	SiftResult(关键点+描述子)	SIFT特征提取
nFeatureMatch	desc1,desc2,method	IntArray(匹配对索引)	特征点匹配
nExtractLBP	gray,w,h,gridX,gridY	FloatArray(LBP直方图)	LBP特征提取
nComputePCA	faces[],w,h,numComponents	PCAResult(均值+特征向量)	PCA特征脸计算
nMeanshift	backProj,w,h>window[]	IntArray(新窗口)	Meanshift跟踪
nCamshift	backProj,w,h>window[]	CamshiftResult(窗口+角度)	Camshift跟踪

6.2 Kotlin API接口规范

类名	方法签名	功能
MotionDetector	detect(gray: Mat): Detection	检测运动目标
ObjectTracker	init(gray: Mat, x: Int, y: Int)	初始化跟踪器
ObjectTracker	track(gray: Mat): TrackResult	执行单帧跟踪
HaarDetector	detect(gray: Mat): List	Haar级联目标检测
HaarDetector	detectNested(gray, nested): Map	多级联嵌套检测
HogDetector	detect(gray: Mat): List	HOG行人检测
FaceDetector	detect(gray: Mat): List	人脸+眼睛检测
FaceRecognizer	enroll(label: String, face: Mat)	注册人脸
FaceRecognizer	recognize(face: Mat): RecognitionResult	识别人脸
FaceDatabase	enroll(label, feature): String	存入人脸特征
FaceDatabase	loadFeature(id: String): FloatArray?	加载人脸特征
VideoStructurer	analyze(videoPath, onProgress): VideoSummary	视频结构化分析

6.3 Fragment路由接口

通过FFViewModel.switchFragment LiveData路由Fragment，菜单层级为：主菜单(已有) -> 图像识别菜单(新增) -> 各功能页面。

第七章 资源与国际化

7.1 资源文件

教材内容	OpenCV函数/类	用途
HOG行人检测	HOGDescriptor.detectMultiScale()	行人检测
SIFT特征点(Ch.11)	SIFT.create()/detectAndCompute()	特征点提取
光流法(Ch.9)	Video.calcOpticalFlowPyrLK()	稀疏光流跟踪
稠密光流(Ch.9)	Video.calcOpticalFlowFarneback()	稠密光流场
MOG2背景建模	Video.createBackgroundSubtractorMOG2()	混合高斯背景减除
KNN背景建模	Video.createBackgroundSubtractorKNN()	KNN背景减除
Meanshift跟踪	Video.meanShift()	概率密度梯度跟踪
Camshift跟踪	Video.CamShift()	自适应窗口跟踪
Shi-Tomasi角点	Imgproc.goodFeaturesToTrack()	光流跟踪特征点
连通域分析	Imgproc.findContours()	前景目标提取
仿射变换	Imgproc.warpAffine()	人脸对齐
SVM分类器(Ch.11)	ml.SVM.create()/train/predict()	物体分类
HSV颜色空间	Imgproc.cvtColor(COLOR_BGR2HSV)	颜色特征提取

第九章 Haar级联模型文件

Haar级联分类器需要预训练的XML模型文件。OpenCV提供了多种预训练模型，可从OpenCV官方仓库获取并打包到项目的assets或raw资源目录中。

模型文件名	检测目标	文件大小	来源
haarcascade_frontalface_default.xml	正脸人脸	~1MB	OpenCV官方
haarcascade_frontalface_alt2.xml	正脸人脸(备选)	~0.5MB	OpenCV官方
haarcascade_profileface.xml	侧脸人脸	~0.5MB	OpenCV官方
haarcascade_eye.xml	眼睛	~0.5MB	OpenCV官方
haarcascade_eye_tree_eyeglasses.xml	眼睛(戴眼镜)	~1MB	OpenCV官方
haarcascade_mcs_nose.xml	鼻子	~0.3MB	OpenCV contrib
haarcascade_mcs_mouth.xml	嘴巴	~0.3MB	OpenCV contrib
haarcascade_fullbody.xml	人体全身	~0.5MB	OpenCV官方
haarcascade_upperbody.xml	人体上半身	~0.5MB	OpenCV官方
haarcascade_lowerbody.xml	人体下半身	~0.5MB	OpenCV官方
haarcascade_car.xml	车辆(正面)	~0.5MB	OpenCV官方

模型文件加载流程：(1) 从assets复制到内部存储目录；(2) CascadeClassifier.load(path)加载；(3) 首次加载缓存路径，后续直接使用。

第十章 测试方案

10.1 单元测试

在imagerecognition模块的src/test/java目录下新增单元测试，覆盖核心算法逻辑的正确性。

测试类	测试方法	测试内容	通过条件
MotionDetectorTest	testFrameDiff	帧差法检测静态帧	motionRatio=0, boxes为空
MotionDetectorTest	testFrameDiffMotion	帧差法检测运动帧	motionRatio>0, boxes非空
MotionDetectorTest	testMOG2	MOG2背景建模	前景掩码正确提取
ObjectTrackerTest	testNCCInit	NCC模板初始化	模板尺寸/位置正确
ObjectTrackerTest	testNCCTrack	NCC单帧跟踪	score>阈值,位置更新
ObjectTrackerTest	testNCCLost	NCC目标丢失	score<阈值,isLost=true
HaarDetectorTest	testLoadCascade	加载级联模型	cascade.load()==true
HaarDetectorTest	testFaceDetect	人脸检测	检测到>=1个人脸框
FaceRecognizerTest	testLBPExtract	LBP特征提取	维度正确(2891维)
FaceRecognizerTest	testEnrollRecognize	注册并识别	label正确,confidence>0.5
FaceDatabaseTest	testEnrollLoad	注册并加载	特征向量一致
FaceDatabaseTest	testDelete	删除人脸	count减1
VideoStructurerTest	testShotDetect	镜头边界检测	突变镜头正确识别
VideoStructurerTest	testKeyFrameExtract	关键帧提取	关键帧数量合理

10.2 集成测试

集成测试在设备上运行，使用项目的测试素材：(1) 静态图像：app/src/main/assets/IMG_20260821_190758.jpg - 测试人脸检测、Haar目标检测、HOG行人检测、形状识别等；(2) 测试视频：app/src/main/assets/video.mp4 - 测试运动目标检测、目标跟踪、视频结构化分析、视频人脸检测等。

10.3 验收标准

功能	验收标准	测试素材
运动目标检测	正确检测视频中运动区域，输出前景掩码和目标框	video.mp4

功能	验收标准	测试素材
目标跟踪	持续跟踪指定目标，轨迹连续无明显跳变	video.mp4
Haar人脸检测	静态图像人脸检测召回率 $\geq 90\%$ ，误检率 $< 10\%$	IMG_20260821_190758.jpg
HOG行人检测	能检测图像中的行人，检测框位置基本准确	IMG_20260821_190758.jpg
人脸识别	注册后能正确识别同一人脸，未注册标记Unknown	手动测试
视频结构化	正确识别镜头边界，提取合理数量关键帧	video.mp4
视频人脸检测	逐帧检测人脸并跟踪，多脸场景不混淆	video.mp4
性能	图像检测 $< 500\text{ms}$ ，视频处理 $\geq 10\text{fps}$	所有素材
UI/UX	页面布局正确，中英文字符串无缺失	手动测试

第十一章 开发计划

阶段	任务	工期	依赖
阶段1	CMakeLists新增OpenCV objdetect/video/ml模块链接	0.5天	无
阶段1	native-lib.cpp新增JNI接口框架(12个方法签名)	0.5天	无
阶段2	MotionDetector.kt实现(帧差/MOG2/KNN)	1天	阶段1
阶段2	ObjectTracker.kt实现(NCC增强+光流)	1.5天	阶段1
阶段2	VideoStructurer.kt实现(镜头检测/关键帧)	1天	阶段1
阶段3	HaarDetector.kt实现+模型文件集成	1天	阶段1
阶段3	HogDetector.kt实现	0.5天	阶段1
阶段3	FeatureMatcher.kt实现(SIFT/ORB)	1天	阶段1
阶段4	FaceDetector.kt实现(人脸+眼睛检测)	1天	阶段3
阶段4	FaceRecognizer.kt实现(LBP+PCA)	2天	阶段3
阶段4	FaceDatabase.kt实现(持久化存储)	0.5天	阶段4
阶段5	JNI C++层实现(各native方法)	3天	阶段2-4
阶段6	8个Fragment页面实现+路由	2天	阶段2-4
阶段6	字符串国际化(中英文120条)	0.5天	阶段6
阶段7	单元测试编写(14个测试类)	1.5天	阶段5
阶段7	集成测试+验收测试	1天	阶段6
阶段8	Bug修复+性能优化	1天	阶段7
	合计	约19天	

第十二章 风险与对策

风险	影响	概率	对策
OpenCV objdetect模块未编译进so	Haar/HOG检测无法使用	低	确认OpenCV编译包含objdetect模块
Haar模型文件过大	APK体积膨胀	中	使用ProGuard压缩或网络下载
视频逐帧检测性能不足	帧率<10fps	中	降分辨率至320x240或跳帧+跟踪
光流法计算量大	帧率下降	中	使用稀疏光流(LK)替代稠密(Farneback)
人脸识别精度不足	误识/拒识率高	中	增加训练样本，调整参数和阈值
OpenCV版本兼容性	API变更编译错误	低	锁定OpenCV版本，查阅4.x文档
JNI内存泄漏	长时间运行OOM	中	确保Mat对象及时release()
多目标跟踪ID切换	跟踪不稳定	中	使用匈牙利匹配优化数据关联

附录

附录A：教材章节与功能映射表

教材章节	对应功能模块	核心算法
第4章 图像分割	MotionDetector(前景提取)	Canny边缘、区域生长、形态学
第5章 压缩编码原理	ObjectTracker(运动估计)	块匹配BMA、运动补偿
第6章 压缩编码标准	ObjectTracker(多精度搜索)	H.264变块运动补偿、1/4像素精度
第9章 质量评价	MotionDetector(光流法)	光流法运动信息提取
第10章 内容检索	VideoStructurer	镜头检测、关键帧、场景聚类
第11章 图像识别	HaarDetector/HogDetector/FaceRecognizer	SVM、CNN理论、Haar特征、统计学习

附录B：已有代码文件清单

文件路径	所属模块	功能
imageCVdeal/.../native-lib.cpp	imageCVdeal	JNI原生实现入口
imageCVdeal/.../CMakeLists.txt	imageCVdeal	CMake构建配置
imageCVdeal/.../OpencvDealJni.kt	imageCVdeal	75个JNI接口方法
imagerecognition/.../NccMatcher.kt	imagerecognition	NCC模板匹配(需增强)
imagerecognition/.../ShapeRecognizer.kt	imagerecognition	Hu不变矩形状识别(需增强)
imagerecognition/.../VideoRecognitionPipeline.kt	imagerecognition	视频识别管线(需增强)

文件路径	所属模块	功能
contentsearch/.../ImageFeatures.kt	contentsearch	HSV颜色直方图+纹理特征
contentsearch/.../ImageSearchEngine.kt	contentsearch	图像检索引擎
app/.../MainActivity.kt	app	Fragment路由
app/.../FFViewModel.kt	app	ViewModel路由控制

附录C：技术栈版本信息

技术	版本	说明
Kotlin	项目配置版本	主开发语言
Java	11	编译兼容版本
C++	C++11	JNI原生层
OpenCV	4.x	libopencv_java4.so动态库
AGP	9.2.1	构建工具
compileSdk	37	编译SDK版本
minSdk	24	最低支持版本(Android 7.0)
CMake	3.22.1	原生构建工具
STL	c++_shared	C++标准库
ABI	arm64-v8a, armeabi-v7a	支持架构